

Trabajo Práctico N°1: Python

Fundación Fulgor

Autor: Diego Maximiliano Villanueva

Docentes: Ariel Pola, Santiago Garaventa Pascual y Agustín Pesce

23 de Mayo de 2026

1. Introducción

El presente trabajo práctico tiene el objetivo de explicar como aborde en Python algunos de los temas vistos durante las primeras clases del cursado, integrando el uso de filtros digitales, comunicación por puerto serie y generación de gráficos. Desarrollé un programa que permite configurar y visualizar un filtro de coseno realzado (Raised Cosine) mediante comandos enviados a través de un puerto serie en modo loopback como pedía la consigna.

2. Estructura del programa

El programa está organizado en un único archivo `TP1.py` que importa la clase `RaisedCosineFilter` desde `filtro.py`. Cuenta con tres funciones auxiliares para generar los distintos gráficos, y un bucle principal que recibe los comandos del usuario, los envía por el puerto serie y los interpreta una vez leídos.

2.1. Comunicación por puerto serie

Se adoptó el enfoque presentado en `com_serie.py`, abriendo el puerto en modo loopback. Cada comando ingresado por teclado se envía codificado por el puerto y se lee de vuelta antes de ser interpretado. En este caso, se usa el modo loopback pero más adelante con la FPGA tendría el mismo funcionamiento solo que habría que cambiar la configuración. Para hacer prueba de esta sección use la librería `time` donde le di un retardo de 2 segundos a la respuesta del puerto serie asegurandome que funcionaba.

3. Comandos implementados

El programa reconoce los siguientes comandos:

- `alpha=<float>`: modifica el atributo `alpha` del filtro (valor entre 0 y 1).
- `span=<int>`: modifica el atributo `span` (entero positivo).
- `sps=<int>`: modifica el atributo `sps` (entero positivo).
- `rrc=<True/False>`: modifica el atributo `rrc`.
- `generate`: regenera los coeficientes del filtro con los parámetros actuales.
- `plot`: invoca el método `plot()` original de la clase.
- `coef`: imprime los coeficientes del filtro.

- **comparar**: grafica tres filtros con distinto valor de **alpha** predefinidos superpuestos.
- **ambas**: muestra la respuesta temporal y en frecuencia en una misma figura.
- **discretizar**: muestra la respuesta temporal con stems (representación discreta).
- **help**: lista los comandos disponibles.
- **exit**: cierra el puerto y termina el programa.

3.1. Manejo de errores

Las conversiones de tipo (`float`, `int`) se encuentran envueltas en bloques `try/except` para evitar que el programa termine inesperadamente si el usuario introduce un valor no numérico. Asimismo, se incluyen validaciones de rango: **alpha** debe estar entre 0 y 1, y **span/sps** deben ser enteros positivos. Esto lo agregue luego de notar que directamente cuando mandaba algo que no sea un número el programa se terminaba al notar el error.

4. Gráficos generados

4.1. Comparación de filtros con distinto alpha

En este caso lo que se busco fue mostrar como variando **alpha** pero manteniendo los valores de **span** y **sps** varía el filtro y su respuesta en el tiempo. Según lo que puedo comprender con un mayor **alpha** se aumenta el ancho de banda que ocupa el filtro pero la respuesta temporal se vuelve mas localizada y no oscila tanto en los laterales. Mientras que para valores más cercanos al 0 se acorta el ancho de banda al mismo tiempo que las oscilaciones laterales se hacen mas prominentes como se puede apreciar en la **Figura 1**.

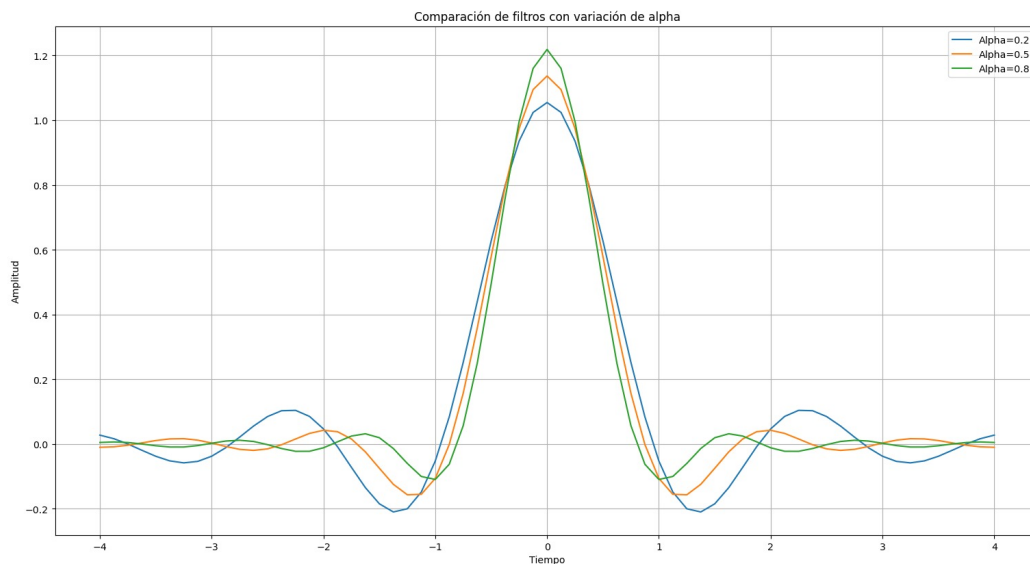


Figura 1: Figura 1: Comparación de filtros con distintos valores de **alpha**.

4.2. Respuesta temporal y en frecuencia

A diferencia del método `plot()` original, que abre dos ventanas independientes, esta función agrupa la respuesta temporal y la respuesta en frecuencia (magnitud en dB) en una sola figura con dos paneles, facilitando la comparación visual entre ambos dominios. Esto se logra usando la función `subplot` de la librería `matplotlib.pyplot`, donde permite dividir la figura en secciones como se puede apreciar en la **Figura 2**.

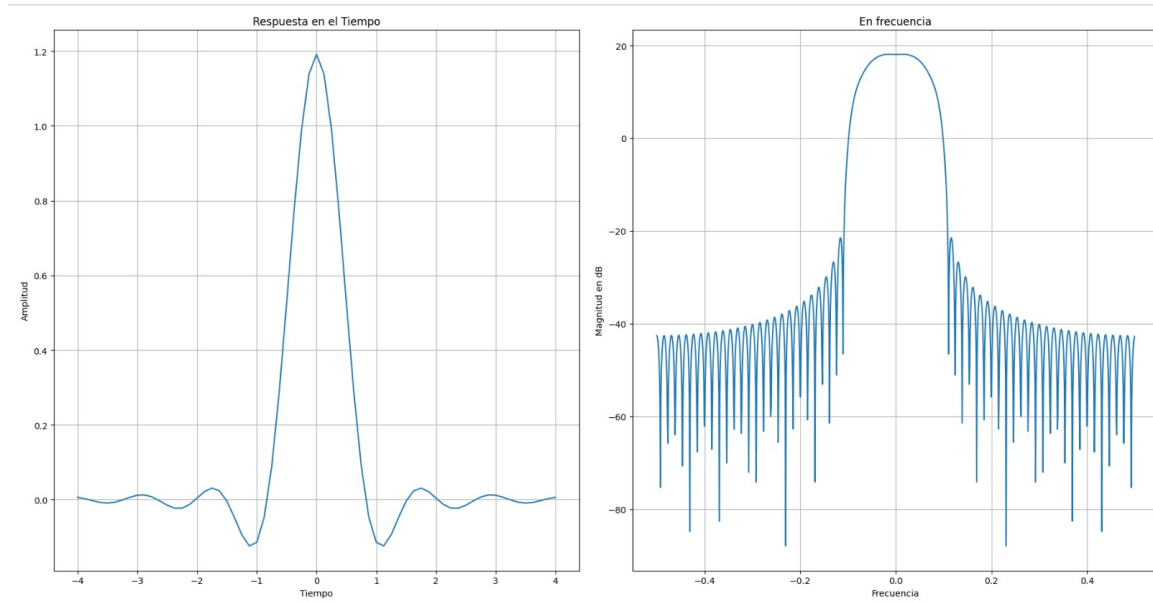


Figura 2: Figura 2: Respuesta temporal y en frecuencia del filtro.

4.3. Respuesta discreta

Este gráfico utiliza `plt.stem()` en lugar de `plt.plot()`, mostrando cada coeficiente como una muestra independiente como se puede apreciar en la Figura 3. Se puede apreciar mejor reduciendo el numero de muestras que viene del cálculo de `span*sps`.

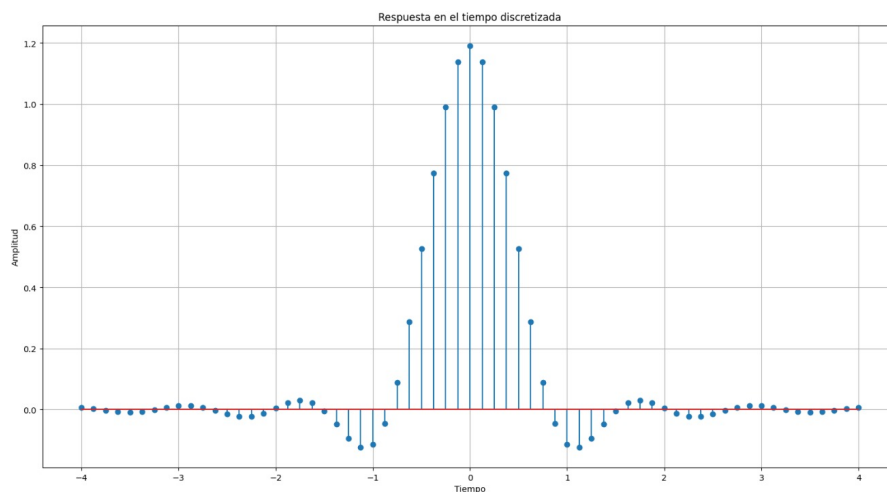


Figura 3: Respuesta del filtro representada de forma discreta con stems.

5. Decisiones de diseño

Durante el desarrollo se tomaron las siguientes decisiones:

- Las funciones para graficar se implementaron como funciones globales dentro de `TP1.py`, en lugar de agregarse como métodos a la clase `RaisedCosineFilter`. Esto evita modificar el

archivo original y permite que las funciones operen sobre múltiples filtros simultáneamente (como en el caso de **comparar**).

- Se utilizan dos variables distintas en el loop principal: **entrada** (lo ingresado por teclado) y **comando** (lo leído del puerto serie). En el modo loopback ambas tienen el mismo contenido, pero la separación deja explícito el flujo y facilita el reemplazo del loopback por una conexión real en caso de querer cambiar a una FPGA.
- Durante el desarrollo sentí que lo mejor era que luego de cada cambio para poder aplicarlo se escriba a mano el comando **generate** ya que me pareció lo óptimo para asegurar que el usuario quiere visualizar este cambio.

6. Conclusiones

El trabajo me permitió comenzar a integrar conceptos de Python que aunque los había visto con anterioridad los tenía abandonados, aparte me ayudó a comprender la comunicación serie. La parte más interesante me parece la representación de figuras con la librería `matplotlib.pyplot` ya que me parece una herramienta con muchas funcionalidades para representar de la mejor manera posible los resultados obtenidos.

Anexo: código fuente

El código completo está disponible en el repositorio: <https://github.com/maxivillanueva7/PROCOM.git> Además a partir de que agregue los primeros comandos comencé a cargar distintos commits para que se registren las distintas funciones que iba agregando.